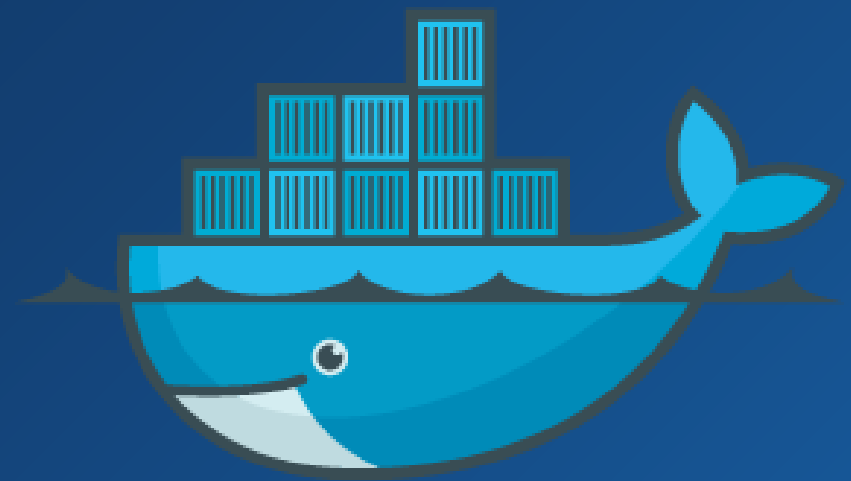


WEEK 8 · SOFTWARE DEVELOPMENT TOOLS AND ENVIRONMENTS

Docker

จากแนวคิด Container สู่การสร้าง Image ใช้เอง
ทฤษฎี + ปฏิบัติการ 4 เล็บ (ผลการรันทั้งหมดในสไลด์นี้ รันจริงบนเครื่อง
เรียน)



| สิ่งที่จะเรียนวันนี้

1 ทำไมต้องใช้ Docker?

The Matrix from Hell และ 3 ปัญหาคลาสสิก

3 Container คืออะไร

การแยกส่วน และการแชร์ OS Kernel

5 Registry / Image vs Container

Docker Hub และความต่างของ image กับ container

L1 คำสั่ง docker run พื้นฐาน

run / ps / pull และวงจรชีวิต container

L3 Volume Mapping

ทำให้ข้อมูลไม่หายเมื่อลบ container

6 สรุปคำสั่งทั้งหมด

ตารางคำสั่งที่ใช้บ่อย ตาราง port และการสร้างระบบ

2 ประวัติของ Docker

จาก cgroups ปี 2008 ถึง Docker Inc.

4 Container vs Virtual Machine

ต่างกันตรงไหน เลือกใช้อย่างไร

S เตรียมเครื่องสำหรับทำแล็บ

เปิด container devtools ที่ติดตั้ง Docker มาให้แล้ว

L2 Port Mapping

เปิดเว็บ Nginx ให้เข้าถึงจากภายนอก

L4 Build Docker Image เอง

Dockerfile ของแอป Node.js

01

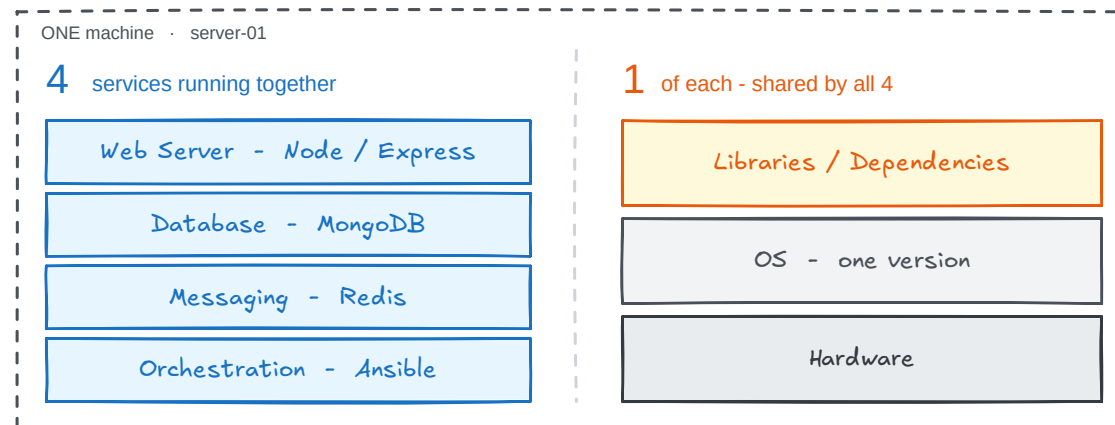
Docker Overview

ทำไมโลกถึงต้องมี Docker — เริ่มจากปัญหาที่ทุกทีมพัฒนาซอฟต์แวร์เคยเจอ

ระบบงานจริงหนึ่งระบบ ประกอบด้วยอะไรบ้าง?

- ▶ ระบบหนึ่งระบบมักมีหลาย **service** ทำงานร่วมกัน
Web Server · Database · Messaging · Orchestration
- ▶ ทุก service ต้องวางอยู่บน **Libraries / Dependencies** ชุดเดียวกัน
- ▶ และต้องใช้ **OS เวอร์ชันเดียวกัน** บนเครื่องเดียวกัน

4 Services, 1 of Everything Else

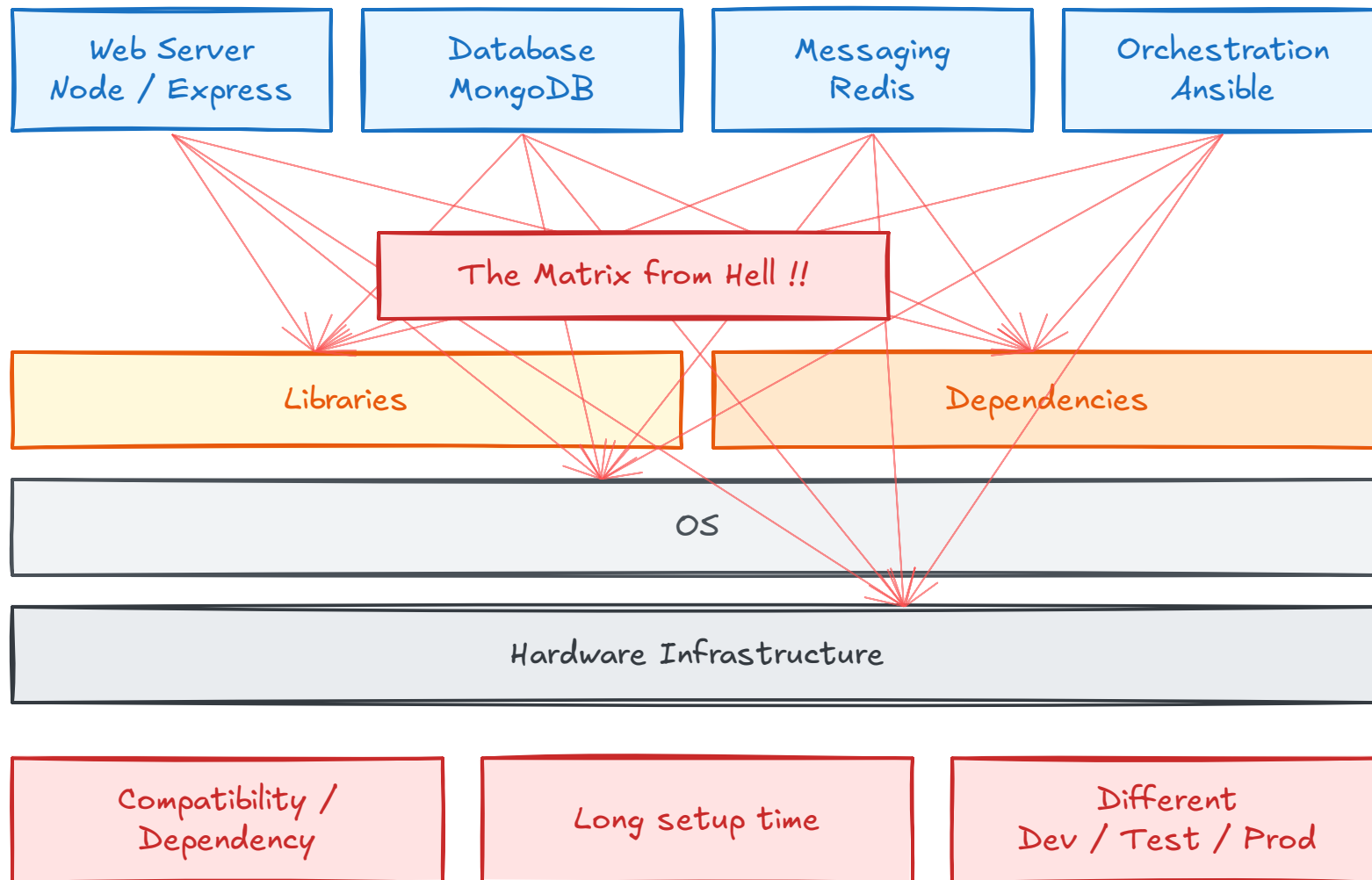


Four different programs, one shared foundation - they all have to agree on it

ทุกอย่างต้อง **เข้ากันได้หมด** — ถ้าตัวใดตัวหนึ่งต้องการเวอร์ชันต่างออกไป ปัญหาจะเริ่มทันที

| ปัญหาที่เกิดขึ้นเมื่อทุกอย่างมัดรวมกัน

เมื่อทุก service ถูกบังคับให้ใช้ Libs และ OS ชุดเดียวกัน



3 ปัญหาคลาสสิกที่ตามมา

1 Compatibility / Dependency

service คนละตัวต้องการ library **คนละเวอร์ชัน** แต่เครื่องเดียวลงได้เวอร์ชันเดียว — ดูตัวอย่างจริงทางขวา

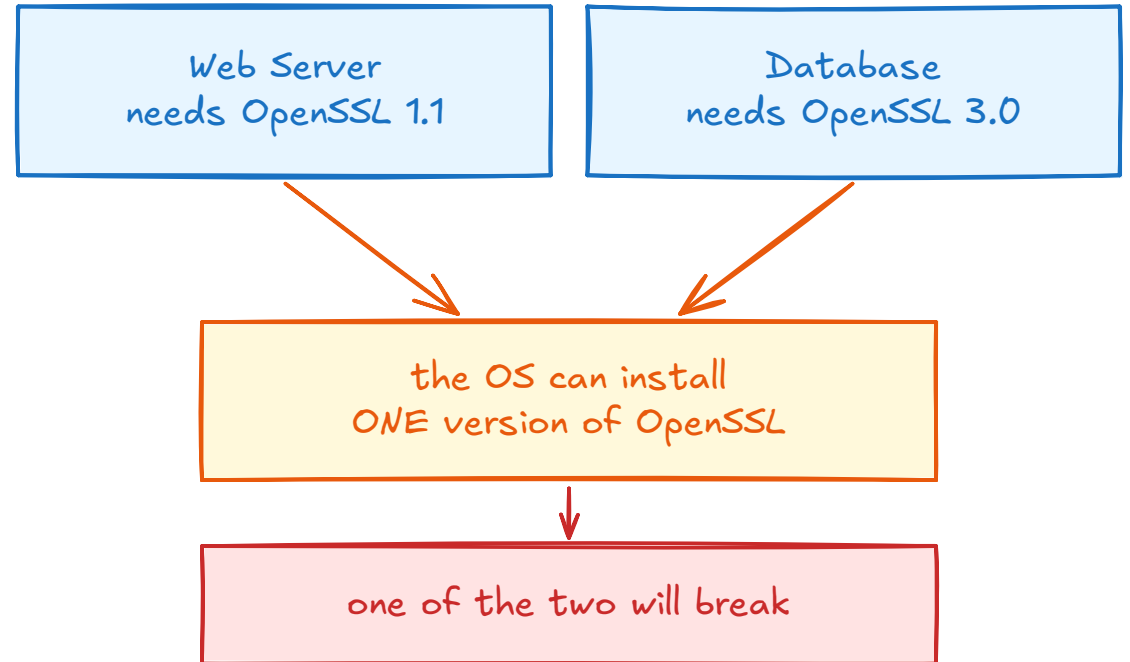
2 Long setup time

เครื่องใหม่ หรือคนใหม่เข้าทีม ต้องไล่ติดตั้งใหม่ทั้งหมด เสียเวลาเป็นวัน และมักตกหล่นบางขั้น

3 Different Dev / Test / Prod

รันได้นบนเครื่องตัวเอง แต่พังบน production — "it works on my machine"

One machine, two versions needed



ต้นเหตุร่วมของทั้ง 3 ข้อคือ **ทุกอย่างมัดรวมอยู่บน OS และ Libs ชุ่เดียวกัน** แยกออกจากกันไม่ได้

Docker คืออะไร?

Docker คือ **open-source platform** ที่ทำให้การ deploy แอปพลิเคชันเป็นอัตโนมัติ โดยรันแอปไว้ใน **software container** — เป็นการจำลอง OS-level virtualization บน Windows และ Linux

Containerize Applications

ห่อแอปพลิเคชัน + ทุกอย่างที่มีมันต้องใช้ ไว้ใน "กล่อง" เดียวที่ยกไป
รันที่ไหนก็ได้

Run each service with its own dependencies

แต่ละ service อยู่คนละ container มี Libs/Deps ของตัวเอง **ไม่**
ชนกัน

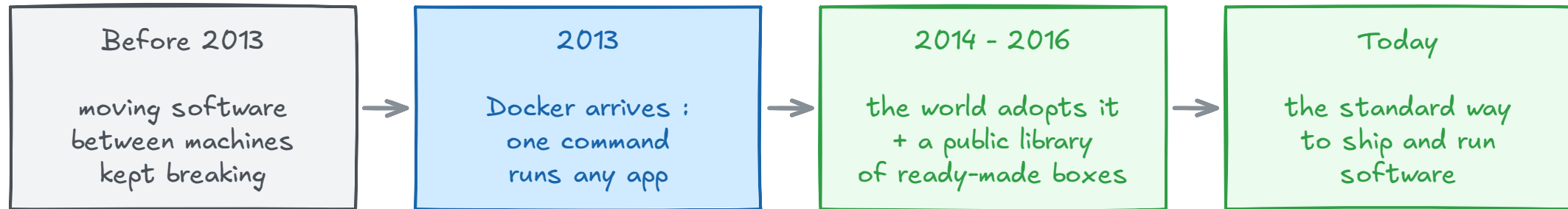
ผลลัพธ์ : เครื่องเดียว รันของที่เวอร์ชันต่างกันได้พร้อมกัน

และย้ายไปเครื่องอื่นแล้วได้ผลเหมือนเดิมทุกครั้ง

ประวัติของ Docker

container ไม่ใช่ของใหม่ — แต่ปี 2013 คือปีที่มันกลายเป็นของที่ทุกคนใช้ได้

The story in four steps



Containers existed long before 2013 - Docker is what made them simple enough for everyone

นิภาพ : ตู้คอนเทนเนอร์บนเรือ

สมัยก่อนขนของขึ้นเรือต้องยกทีละกระสอบทีละถัง รูปทรงไม่เหมือนกันสักอัน ซ้ำและของเสียหายง่าย พอโลกตกลงใช้ **ตู้ขนาดมาตรฐานเดียวกัน** เคนเรือ รถบรรทุก ก็ยกตู้ไหนก็ได้ — Docker ทำแบบเดียวกันกับซอฟต์แวร์ (ที่มาของชื่อและโลโก้เรือ)

ปัญหาก่อนปี 2013

โปรแกรมที่รันได้บนเครื่องคนเขียน พอย้ายไปเครื่องอื่นมัก **พัง** เพราะเวอร์ชันของ library และ OS ไม่เหมือนกัน การติดตั้งใหม่แต่ละครั้งใช้เวลาเป็นวัน และมักตกหล่นบางชิ้น

Docker เปลี่ยนอะไร

เอาโปรแกรม **บวกกับทุกอย่างที่มันต้องใช้** ใส่กล่องเดียวกัน แล้วสั่งรันด้วยคำสั่งเดียว `docker run` — ยกกล่องนั้นไปวางเครื่องไหนก็ได้ผลเหมือนเดิมทุกครั้ง

แล้ว Docker เปลี่ยนอะไร?

งานเดียวกัน — เทียบก่อนกับหลังมี Docker

ก่อนมี Docker

1. ติดตั้งแพ็คเกจของ OS ให้ถูกต้อง
2. ติดตั้งโปรแกรมและ library ทั้งหมด
3. ไล่แก้เวอร์ชันที่ชนกันด้วยมือ
4. ตั้งค่าให้ครบทุกอย่าง
5. ทำซ้ำทั้งหมดนี้อีกรอบบนเครื่องถัดไป

ใช้เวลาเป็นชั่วโมง · พอเปลี่ยนเครื่องก็พังอีก

หลังมี Docker

ทั้งหมดที่ต้องทำ เหลือแค่บรรทัดเดียว :

```
docker run nginx
```

→ ได้เว็บเซิร์ฟเวอร์ที่ทำงานอยู่แล้ว ไม่ต้องติดตั้งอะไรเพิ่ม

ใช้เวลาไม่กี่วินาที · ได้ผลเหมือนกันทุกเครื่อง

Dockerfile

ไฟล์สูตรสร้างกล่อง เขียนครั้งเดียว
ใครเอาไปสร้างก็ได้ของเหมือนกัน

Image เป็นชั้น ๆ

ชั้นที่ซ้ำกับของเดิมไม่ต้องโหลดใหม่
ประหยัดทั้งพื้นที่และเวลา

Docker Hub

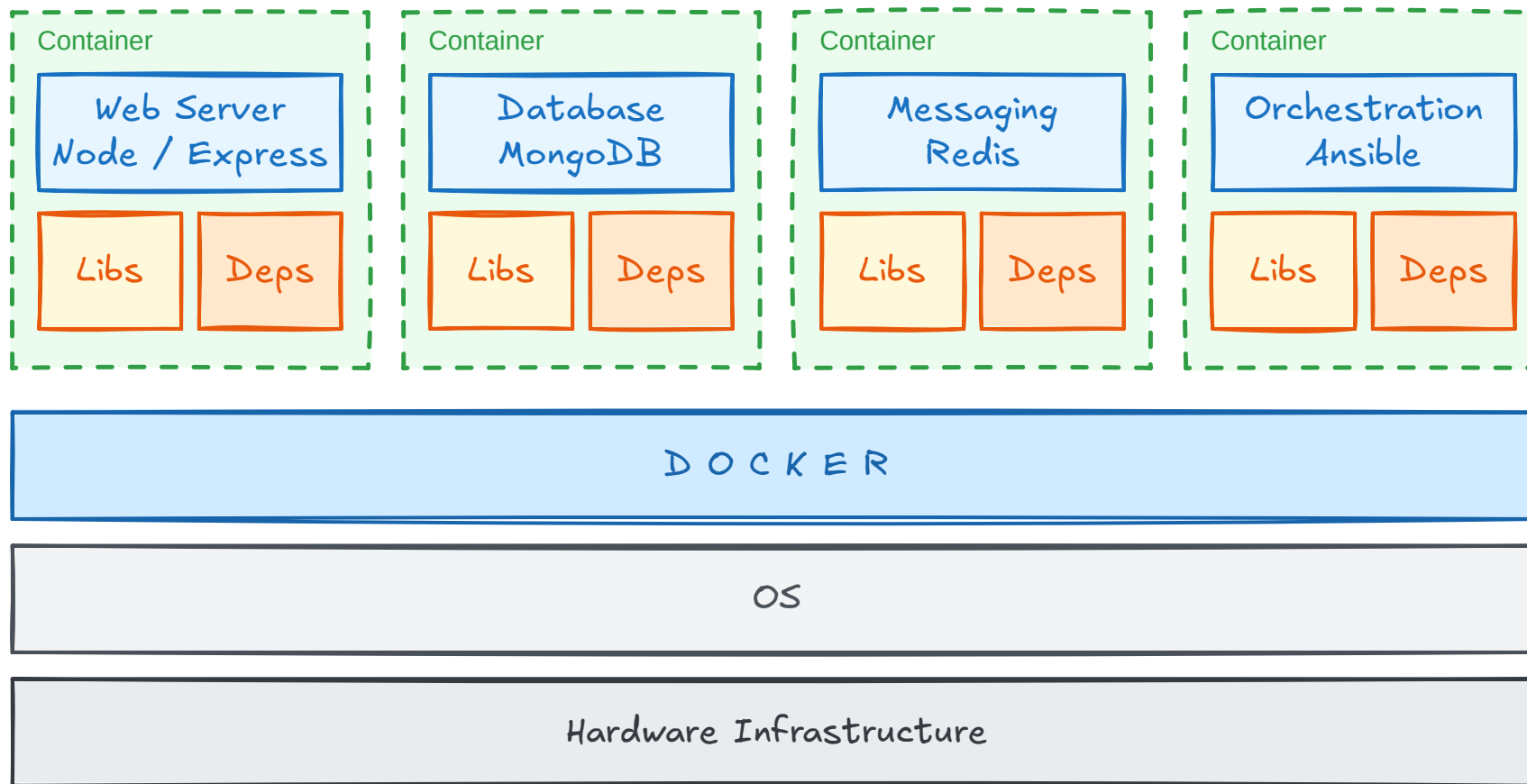
คลังกลาง มีของสำเร็จรูปให้ดึงมาใช้ได้
ทันที ไม่ต้องสร้างเอง

คำสั่งเดียวจบ

docker run แค่นี้ก็ได้ของที่ทำงาน
อยู่จริง ไม่ต้องตั้งค่าอะไรอีก

ภาพออก : แยกทุก service ลงคนละ Container

Docker = Containerize Applications

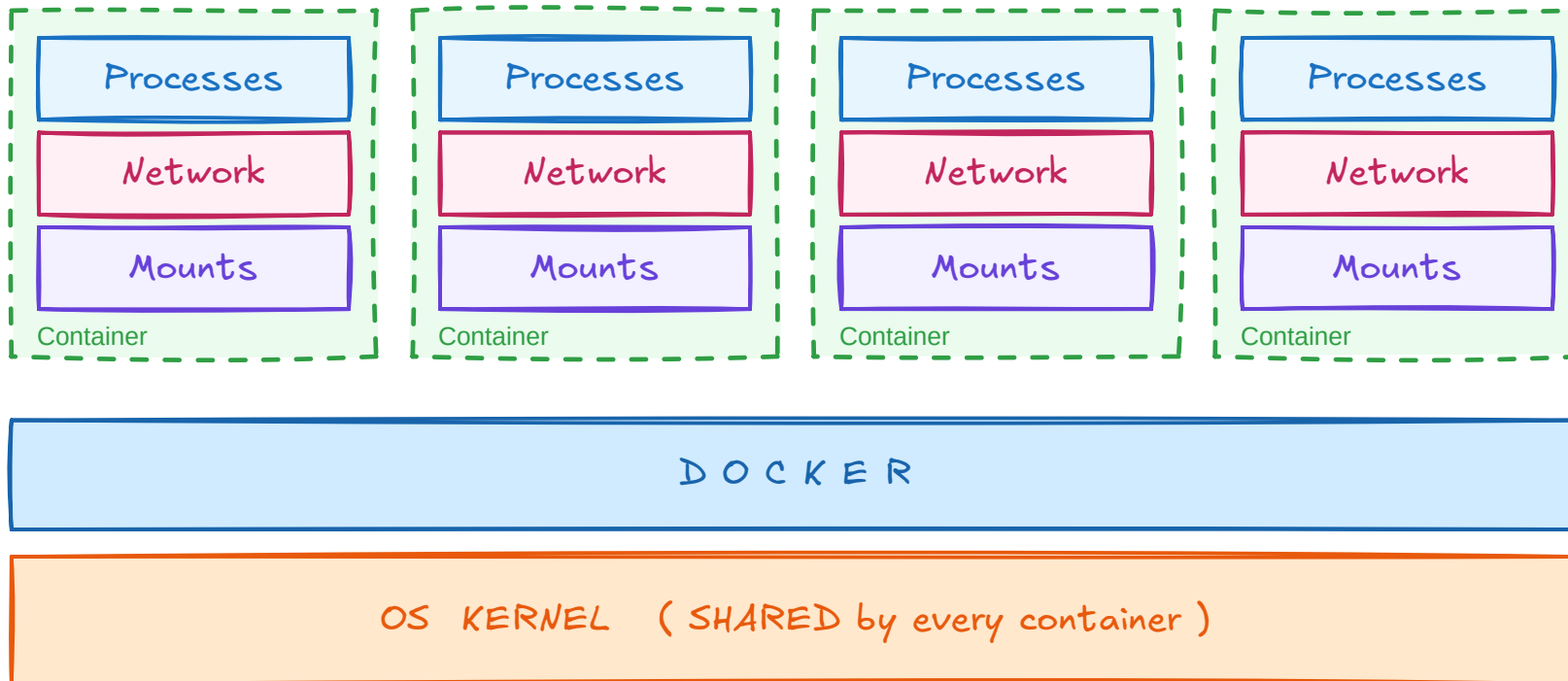


Each service brings its OWN Libs + Deps -> no crossing lines, no conflicts

Container คืออะไร?

สภาพแวดล้อมที่ถูกแยกออกจากกัน แต่ยังใช้ kernel ร่วมกัน

What are Containers?



Isolated: own Processes / Network / Mounts Shared: ONE kernel -> small + fast



โลโก้ Docker คือวาฬที่แบกตู้คอนเทนเนอร์ — ตัววาฬคือ Docker ที่รันอยู่บนเครื่อง ส่วนตู้บนหลังคือ container แต่ละตัวที่มันแบกไว้ ตรงกับรูปทางซ้ายพอดี

อยู่ในกล่องแล้วมองเห็นอะไรบ้าง?

3 อย่างที่แต่ละกล่องมีเป็นของตัวเอง — ลองสั่งคำสั่งดูได้จริง

1. โปรแกรมที่รันอยู่

สั่งดูรายชื่อโปรแกรมในกล่อง จะเห็นแค่ของ
กล่องตัวเอง ไม่เห็นของกล่องอื่น และไม่เห็นของ
เครื่องแม่

```
docker run --rm busybox ps
```

2. เครือข่ายและเลข IP

แต่ละกล่องมี IP ของตัวเอง — boxA เป็น
172.18.0.2 boxB เป็น 172.18.0.3 ทั้งคู่
จึงเปิด port 80 พร้อมกันได้โดยไม่ชนกัน

3. ไฟล์และโฟลเดอร์

สั่งดูไฟล์ในกล่อง จะเห็นไฟล์ของกล่องนั้น ไม่ใช่
ไฟล์ในเครื่องเรา เว้นแต่เราเอาไปแปะให้เอง
ด้วย -v

```
docker exec boxA ls /
```

ทั้ง 3 อย่างนี้ **Linux ทำให้อยู่แล้ว** — Docker แค่สั่งให้มันทำงานให้อัตโนมัติ ไม่ได้สร้าง OS ใหม่ซ้อนเข้าไป

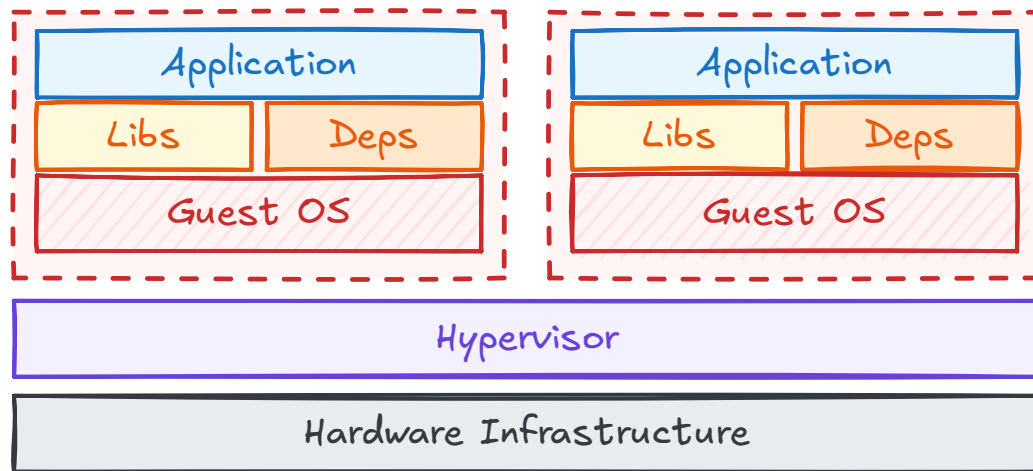
ไม่มี OS ซ้อน → กล่องจึง เล็กและเริ่มทำงานเร็ว

เทียบโครงสร้างทีละชั้น

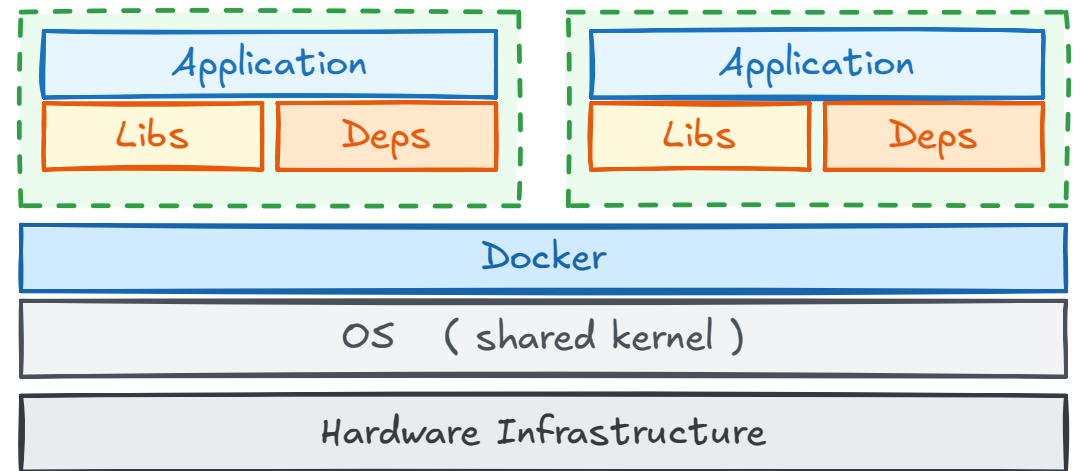
จุดต่างอยู่ที่ชั้นกลาง — VM ทุกตัวแบก OS ของตัวเอง แต่ Container ไม่มี

Containers vs Virtual Machines

Virtual Machine



Container (Docker)



Size

VM



Boot up

VM



Where the resources go

VM



Container



Container



Container



Same machine, same total resources - a VM spends much of it on its own OS, a container does not. Bars are illustrative, not measured.

ตารางเปรียบเทียบ

	Virtual Machine	Container (Docker)
โครงสร้าง อ่านจากล่างขึ้นบน	Application Libs / Deps Guest OS — ทุก VM ต้องมี Hypervisor Hardware	Application Libs / Deps Docker — ไม่มี Guest OS OS (แชร์ kernel) Hardware
Size	ระดับ GB	ระดับ MB
Boot up	ช้า (เป็นนาที)	เร็ว (เป็นวินาที)
Utilization	ใช้ทรัพยากรมาก เพราะมี OS หลายชุดซ้อนกัน	ใช้ทรัพยากรน้อย เพราะแชร์ kernel เดียว
Isolation	แยกสมบูร์ณระดับ OS	แยกระดับ process

ยิ่งแยกสมบูร์ณ ยิ่งกินทรัพยากร — VM ปลอดภัยกว่าในแง่การแยก แต่ Container เบากว่าและเร็วกว่ามาก

| ในทางปฏิบัติ : ใช้ร่วมกันได้

VM และ Container ไม่ใช่คู่แข่งกันเสมอไป

รูปแบบที่พบบ่อยที่สุด

- มีเครื่องหนึ่งเครื่อง (เครื่องจริง หรือ VM ก็ได้)
- ติดตั้ง **Docker** ไว้ข้างใน
- รันหลาย container บนเครื่องนั้น

ได้ประโยชน์ทั้งสองทาง

เครื่อง/VM ให้การแยกระดับ OS และการจัดสรรทรัพยากร

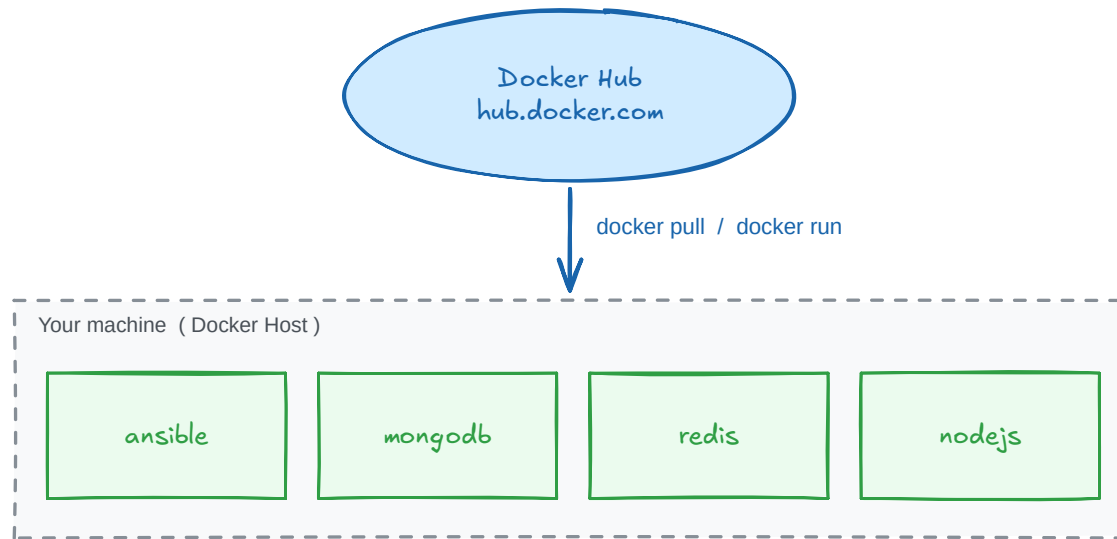
Container ให้ความเบา ความเร็ว และความสม่ำเสมอของสภาพแวดล้อม

แล็บในวิชานี้ก็เป็นรูปแบบเดียวกัน — เรามีเครื่องเรียนคือ container devtools ที่มี Docker อยู่ข้างใน แล้วรัน container ของแล็บซ้อนอยู่ในนั้นอีกที (Docker-in-Docker) ทั้งหมด**อยู่บนเครื่องเราเอง ไม่ได้ใช้ cloud**

image ที่เราใช้ มาจากไหน?

คลังเก็บ image กลางที่ทุกคนดึงไปใช้ได้

Docker Registry (Docker Hub)



docker run <image> -> not found locally? Docker pulls it from the registry first

docker pull – โหลดมาเก็บไว้เฉย ๆ ยังไม่รัน

```
docker pull ansible
docker pull mongodb
docker pull redis
docker pull nodejs
```

docker run – โหลดให้ถ้ายังไม่มี แล้วรันต่อทันที

```
docker run ansible
docker run mongodb
docker run redis
docker run nodejs
```


Registry ใช้ได้สองทาง

ดึงของคนอื่นลงมาใช้ และส่งของเราขึ้นไปแบ่งปัน

ดึงลงมาใช้ — pull

ของสำเร็จรูปมีให้เกือบทุกอย่าง ดึงมาใช้ได้เลย **ไม่ต้องติดตั้งเอง**

```
docker pull nginx
```

ส่งขึ้นไปแบ่งปัน — push

image ที่เราสร้างเอง ส่งขึ้นคลัง ให้เพื่อนหรือเซิร์ฟเวอร์อื่นดึงไปใช้
ต่อได้

```
docker push myname/myapp:1.0
```

คลังกลางที่เป็น**ค่าเริ่มต้น**คือ **Docker Hub** (hub.docker.com) — ถ้าไม่ระบุที่อื่น Docker จะไปหาที่นี้เสมอ

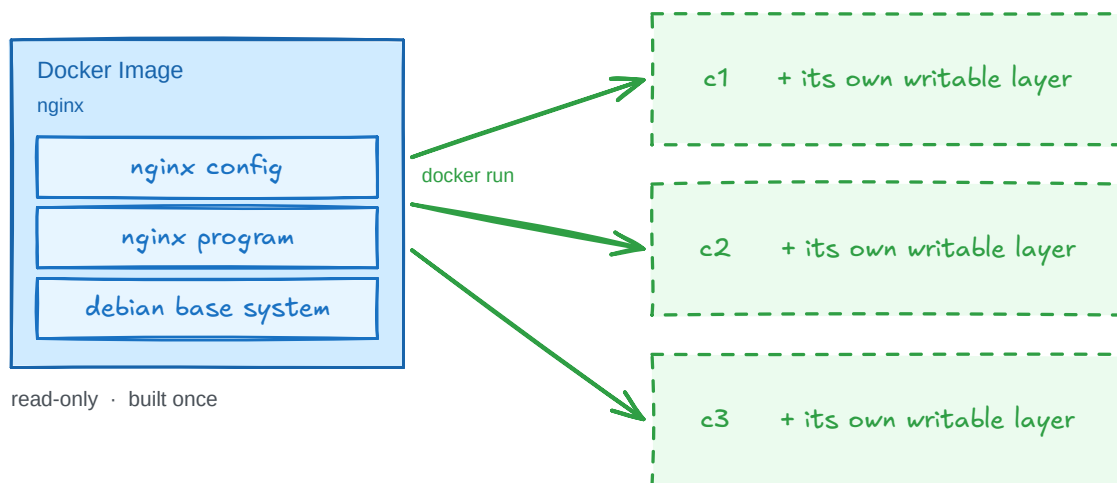
รันทีตัวก็ได้ แต่ละตัวเป็น container แยกกัน

ดึง image เดิมมาใช้ซ้ำได้ไม่จำกัด

1 image สร้าง container ได้ไม่จำกัด

พิมพ์ชื่อ image เดิมซ้ำ ๆ ก็ได้ container เพิ่มทีละตัว

One Image -> Many Containers



The read-only image is SHARED - each container only adds its own thin writable layer

สั่ง 3 ครั้ง จาก image ตัวเดียวกัน

```
docker run -d --name c1 nginx
docker run -d --name c2 nginx
docker run -d --name c3 nginx
```

ทั้ง 3 บรรทัดพิมพ์ชื่อ image **เดียวกันคือ nginx** — ต่างกันแค่ --name เท่านั้น

แล้วนับดู — ผลจริงจริง

```
docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
busybox:latest	dc2d74b28e4c	6.81MB	2.23MB	U
nginx:latest	8541484afbc9	241MB	66MB	U
ubuntu:latest	678c6550cc43	160MB	45.3MB	U


```
docker ps # (ตัดบางคอลัมน์)
```

CONTAINER ID	IMAGE	STATUS	PORTS	NAMES
a10a0f6244d3	nginx	Up Less than a second	80/tcp	c3
0b44408571c9	nginx	Up Less than a second	80/tcp	c2
493876193c31	nginx	Up 1 second	80/tcp	c1

→ image **1 ตัว** · container **3 ตัว** · CONTAINER ID ไม่ซ้ำกันเลย

Image กับ Container ต่างกันอย่างไร?

เทียบกับของใกล้ตัว แล้วดูจากคำสั่งจริง

Image = ไฟล์ติดตั้งที่เก็บไว้ในเครื่อง

เหมือนไฟล์ติดตั้งโปรแกรมที่โหลดมาเก็บไว้ แต่ยังไม่ได้อ่าน

- ตัวมันไม่เปลี่ยนแปลงเอง
- ก็ปส่งให้เพื่อนแล้วได้ของเหมือนกันปะ

Container = โปรแกรมที่กำลังเปิดอยู่

พอสั่งเปิด ก็ได้ของที่ ทำงานอยู่จริง

- เปิดพร้อมกันหลายตัวจากไฟล์เดียวได้
- สิ่งที่เกิดขึ้นตอนเปิด จะหายไปเมื่อลบทิ้ง

```
docker build -t myapp:1.0 .    # ได้ image 1 อัน
docker images                  # เห็น myapp:1.0
```

```
docker run --name a myapp:1.0  # container ตัวที่ 1
docker run --name b myapp:1.0  # ตัวที่ 2 จาก image เดิม
docker ps                       # เห็นทั้ง a และ b
```

ลบ container a ทิ้ง — **image ยังอยู่** สร้างตัวใหม่จาก image เดิม
ได้เสมอ

Image = แม่พิมพ์ • **Container = ของที่ปั๊มออกมา**

แม่พิมพ์อันเดียว ปั๊มของออกมาได้ไม่จำกัดจำนวน

SETUP

เครื่องสำหรับทำแล็บ

ใช้ image `tuchsanai/devtools` ที่ ติดตั้ง Docker มาให้เรียบร้อยแล้ว
ข้ามขั้นตอนติดตั้งไปเริ่มใช้งานได้ทันที

เปิดเครื่องสำหรับทำแล็บ

3 คำสั่งบนเครื่องของเรา แล้วเข้าไปทำแล็บข้างใน

1) ลบเครื่องเรียนตัวเดิมทิ้งก่อน (ถ้าเคยสร้างไว้)

```
docker rm -f devtools
```

2) สร้างเครื่องเรียนขึ้นมาใหม่ และเปิด SSH ที่ port 2222

```
docker run -dit --name devtools --privileged -p 2222:22 tuchsanai/devtools:2569_1
```

3) เข้าไปในเครื่องเรียนผ่าน SSH (password : passwd)

```
ssh root@localhost -p 2222
```

-dit

-d รันเบื้องหลัง · -i เปิด stdin ค้างไว้ · -t ให้มี terminal — รวมกันแล้วกล่องจะไม่ดับทันที

--privileged

ให้สิทธิ์เต็ม เพื่อให้ **รัน Docker ซ่อนอยู่ข้างในกล่อง** ได้ (Docker-in-Docker) — จำเป็นสำหรับแล็บนี้

-p 2222:22

ส่ง port 2222 ของเครื่องเรา เข้า port 22 (SSH) ของกล่อง — เรื่องนี้จะอธิบายละเอียดใน LAB 2

ใน **VS Code** ใช้ส่วนขยาย **Remote-SSH** ต่อไปที่ root@localhost:2222 จะได้หน้าต่างที่ทำงาน**อยู่ในเครื่องเรียน**โดยตรง — ทั้ง terminal และการแก้ไขไฟล์ (แล็บทั้งหมดของสัปดาห์นี้ทำบนเครื่องเรา ไม่ใช่ cloud)

ตรวจสอบว่า Docker พร้อมใช้งาน

รันจริงในเครื่องเรียน devtools — ผลลัพธ์ที่ได้จริง

```
docker --version
```

```
Docker version 29.6.2, build dfc4efb
```

```
docker compose version
```

```
Docker Compose version v5.3.1
```

ถ้าทั้งสองคำสั่งขึ้นเลขเวอร์ชัน = **พร้อมทำแล็บ** ไม่ต้องติดตั้งอะไรเพิ่ม

สังเกต : เป็น docker compose (เว้นวรรค) ไม่ใช่ docker-compose (ขีดกลาง) — แบบขีดกลางคือรุ่นเก่าที่เลิกใช้แล้ว

LAB 1

Docker Run

คำสั่งพื้นฐานที่ใช้บ่อยที่สุด — `run` · `ps` · `pull`
และวงจรชีวิตของ container

docker run – สร้างและเริ่ม container

```
docker run nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
5a4222b844e8: Pulling fs layer
c0df8d325117: Pulling fs layer
... (รวม 7 layer) ...
26c307b5e35a: Pull complete
3c55dc422a81: Pull complete
5a4222b844e8: Pull complete
Digest: sha256:8541484afbc9c8a5a8a99b379568ebbc957f658583ec9448fc43104229c03cf8
Status: Downloaded newer image for nginx:latest
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/08/10 15:02:34 [notice] 1#1: nginx/1.31.3
2026/08/10 15:02:34 [notice] 1#1: start worker processes
^ ค้างอยู่ตรงนี้ – container ยังทำงาน กด Ctrl+C เพื่อหยุด
```

ถ้า **ไม่มี image ในเครื่อง** Docker จะ **pull ให้อัตโนมัติ** จาก registry แล้วค่อยรัน – ไม่ต้องสั่ง pull เองก่อน

รันแบบนี้เป็น **foreground** – terminal จะค้างและพ่น log ออกมาเรื่อย ๆ ถ้าอยากได้ prompt คืน ให้เติม -d

อ่านผลลัพธ์ให้เป็น

Image ถูกดึงมาเป็น Layer

5a4222b844e8, c0df8d325117, f5de6e85ac74 ... คือ layer ย่อยของ image (nginx มี 7 layer)

Pull complete = โหลดใหม่

Already exists = มีอยู่แล้ว ไม่ต้องโหลดซ้ำ

Digest คืออะไร

ค่า sha256:8541484afbc9... คือลายนิ้วมือของ image เวอร์ชันนั้นเป๊ะ ๆ

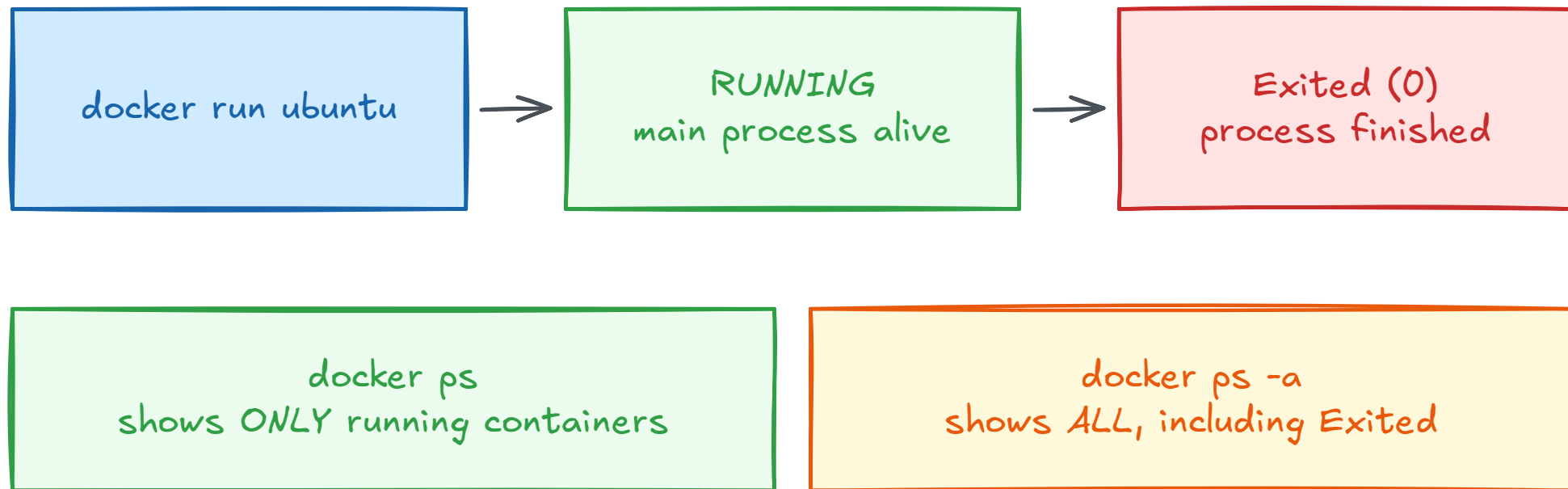
ใช้ยืนยันว่าที่โหลดมาคือของชิ้นเดียวกับต้นทางจริง ไม่ถูกแก้ไขระหว่างทาง

ประโยชน์ของ layer : หลาย image ที่สร้างจากฐานเดียวกันจะ **ใช้ layer ร่วมกัน** — ประหยัดพื้นที่ดิสก์และเวลาดาวน์โหลดมาก

ตัวเลข digest ของแต่ละคนอาจไม่ตรงกับสไลด์ เพราะ nginx:latest ถูกอัปเดตอยู่เรื่อย ๆ — ค่าที่เห็นนี่คือของวันที่รันจริง

วงจรชีวิตของ Container

Container Lifecycle = life of its MAIN process



`docker run ubuntu` -> `/bin/bash` ends immediately -> container Exits at once

`docker run ubuntu sleep 5` -> stays alive for 5 seconds, then Exits

วงจรชีวิต — ลองสิ่งจริงทีละขั้น

stop → start → rm ดูค่า STATUS เปลี่ยนไปทีละขั้น

```
docker run -d --name c1 nginx
```

```
docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
0520924d411b	nginx	"/docker-entrypoint...."	Less than a second ago	Up Less than a second	80/tcp	c1

```
docker stop c1
```

```
docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
0520924d411b	nginx	"/docker-entrypoint...."	1 second ago	Exited (0) Less than a second ago		c1 # หยุดแล้ว แต่ยังมีอยู่

```
docker start c1
```

```
docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
0520924d411b	nginx	"/docker-entrypoint...."	1 second ago	Up Less than a second	80/tcp	c1 # กลับมาทำงานต่อได้

```
docker rm -f c1
```

```
docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
						# เหลือแค่หัวตาราง — ถูกลบออกจริง

stop ≠ rm — stop แค่หยุด container ยังอยู่และ start กลับมาได้ ส่วน rm คือลบทิ้งถาวร

docker ps เห็นเฉพาะตัวที่ **กำลังทำงาน** — พอ stop แล้วต้องใช้ docker ps -a ถึงจะเห็น

docker ps vs docker ps -a

```

docker run ubuntu      # ubuntu ไม่มีคำสั่งค้างไว้ จึงจบทันที
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
a7fb98a8eddd: Pull complete
617772c7d19b: Pull complete
Digest: sha256:678c6550cc43645e08669028bc177f50be4e7c5b8cca677067b1914d4afc7a03
Status: Downloaded newer image for ubuntu:latest

docker ps              # แสดงเฉพาะที่ "กำลังทำงาน" → เหลือแค่หัวตาราง
CONTAINER ID    IMAGE    COMMAND    CREATED    STATUS    PORTS    NAMES

docker ps -a          # แสดงทั้งหมด รวมทั้งที่หยุดไปแล้ว
CONTAINER ID    IMAGE    COMMAND    CREATED    STATUS    PORTS    NAMES
cbf31d44bfbl    ubuntu    "/bin/bash"    1 second ago    Exited (0) Less than a second ago    loving_ride

```

บทเรียนสำคัญ : container มีชีวิตอยู่ตราบเท่าที่ **process หลักในนั้นยังทำงาน** — พอ /bin/bash จบ container ก็ Exited (0) กันที ไม่ใช่ว่ามัน "พัง" แต่มัน "ทำงานเสร็จแล้ว"

เลข (0) คือ exit code — **0 = จบปกติ** · ชื่อ loving_ride คือชื่อสุ่มที่ Docker ตั้งให้ เพราะเราไม่ได้ใส่ --name

สั่งคำสั่งให้ container ทำ

```
docker run busybox echo hi there
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
b05093807bb0: Pull complete
Digest: sha256:dc2d74b28e4cf8984fa52af1f39bc7c3d9c73760b41a74d629f5d11b1ab28616
Status: Downloaded newer image for busybox:latest
hi there
```

<- ผลของคำสั่ง echo ที่เราต่อท้าย

รูปแบบ: `docker run <image> <command>` — คำสั่งที่ต่อท้ายจะไป **แทนที่คำสั่งเริ่มต้น** ของ image นั้น

busybox เป็น image จี๋วมาก — วัดจริงด้วย `docker images` ได้ **2.23 MB** (เทียบกับ nginx 66 MB, ubuntu 45.3 MB) จึงนิยมใช้ทดสอบเพราะโหลดเร็ว

ต่อท้ายคำสั่ง (Append a command)

```
docker run ubuntu          # จบหันที่ ได้ prompt คืนเลย

docker run ubuntu sleep 5  # ไม่มี output แต่ terminal ค้างไป 5 วินาที
                           # แล้วค่อยได้ prompt คืน – container มีชีวิตอยู่ตอนนั้น

docker run ubuntu sh -c "echo Hello && echo World && ls && pwd && date"
Hello
World
bin boot dev etc home lib lib64 media mnt opt proc # <- ผลจริงพิมพ์บรรทัดละรายการ
root run sbin srv sys tmp usr var                  # ย่อให้อยู่ 2 บรรทัดเพื่อประหยัดพื้นที่
/
Mon Aug 10 15:26:11 UTC 2026
```

ผลลัพธ์ของ `ls` คือ **ไฟล์ระบบของ Ubuntu ที่อยู่ในกล่อง** ไม่ใช่ของเครื่องเรา — นี่คือ **Mount Namespace** ที่แยกกันตามที่พูดไปตอนต้น

ใช้ `sh -c "..."` เมื่อต้องการรันหลายคำสั่งต่อกัน เพราะ `&&` เป็นไวยากรณ์ของ shell ไม่ใช่ของ docker

docker pull – ดาวน์โหลดอย่างเดียว

```
docker pull nginx          # ครั้งแรก : ยังไม่มีในเครื่อง
Using default tag: latest
latest: Pulling from library/nginx
5a4222b844e8: Pulling fs layer ... (รวม 7 layer) ...
26c307b5e35a: Pull complete ... (ครบทั้ง 7 layer) ...
Digest: sha256:8541484afbc9c8a5a8a99b379568ebbc957f658583ec9448fc43104229c03cf8
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest

docker pull nginx          # ครั้งที่สอง : มีอยู่แล้ว ไม่โหลดซ้ำ
Using default tag: latest
latest: Pulling from library/nginx
Digest: sha256:8541484afbc9c8a5a8a99b379568ebbc957f658583ec9448fc43104229c03cf8
Status: Image is up to date for nginx:latest # <- ไม่มีบรรทัด layer เลย
docker.io/library/nginx:latest
```

docker pull

โหลด image มาเก็บไว้ในเครื่องเลย ๆ **ไม่รัน**

docker run

โหลด (ถ้ายังไม่มี) แล้วรันต่อทันที

Using default tag: latest — ถ้าไม่ระบุ tag Docker เติม : latest ให้เสมอ ในงานจริงควรระบุเวอร์ชันชัดเจน เช่น nginx:1.31

docker images – ดูของที่มีในเครื่อง

หลังทำ LAB 1 จบ เครื่องเรามี image อยู่ 3 ตัว

```
docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
busybox:latest	dc2d74b28e4c	6.81MB	2.23MB	U
nginx:latest	8541484afbc9	241MB	66MB	
ubuntu:latest	678c6550cc43	160MB	45.3MB	U

CONTENT SIZE

ขนาดที่ **ดาวน์โหลดมาจริง** (ถูกบีบอัดไว้) —
nginx ทั้งตัวแค่ **66 MB**

DISK USAGE

ขนาดที่ **กินพื้นที่จริงบนดิสก์** หลังแตกไฟล์ออกมาแล้ว

EXTRA = U

U = in Use มี container ใช้ image นี้อยู่ (ลบ container หมดเมื่อไร ตัว U ก็หายไป)

เทียบกับ VM : Ubuntu Server เต็มตัวเป็น image ขนาดหลาย **GB** แต่ ubuntu ใน Docker แค่ **45.3 MB** — เพราะไม่มี kernel และ service ของ OS ติดมาด้วย

LAB 2

Port Mapping

ทำให้เว็บที่รันอยู่ในกล่อง เข้าถึงได้จากเบราว์เซอร์ภายนอก

ปัญหา : container มี IP ของตัวเอง

เข้าถึงไม่ได้จากภายนอก

container ได้ IP ภายใน — บนเครื่องเรียนนี้รันจริงได้
172.18.0.2, 172.18.0.3, ...

IP นี้ เข้าถึงได้เฉพาะจาก **Docker Host** เท่านั้น — เครื่องอื่นในเครือข่ายมองไม่เห็น

ทางแก้ : Port Mapping

จอง port บนเครื่อง host แล้วบอก Docker ว่า
"ใครมาที่ port นี้ ให้ส่งต่อเข้า port นั้นในกล่อง"

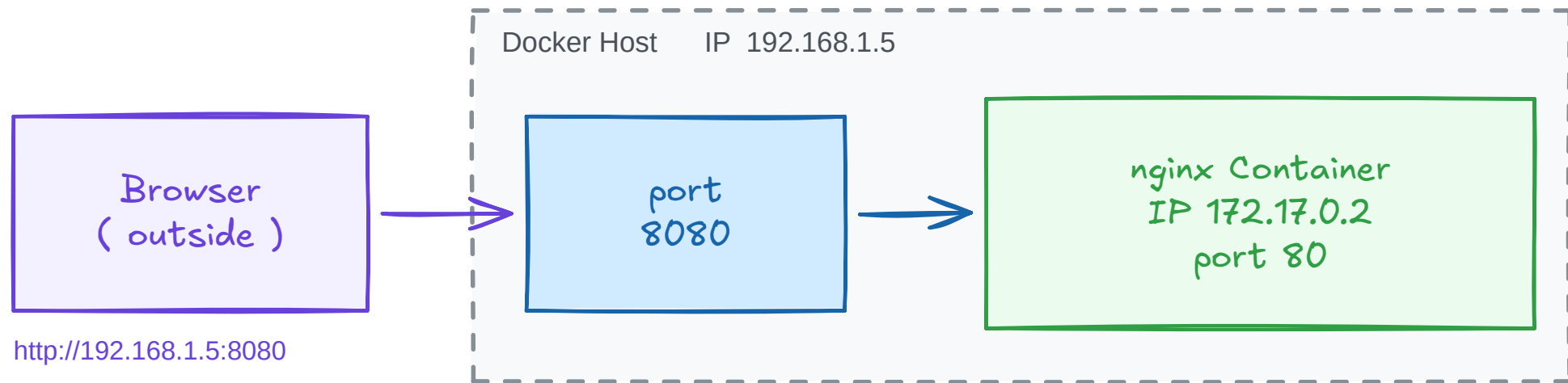
```
docker run -p <HOST_PORT>:<CONTAINER_PORT> <image>
```

HOST_PORT port ฝั่งเครื่องเรา

CONTAINER_PORT port ที่แอปฟังอยู่ในกล่อง

Port Mapping ทำงานอย่างไร

Port Mapping -p HOST:CONTAINER



```
docker run -p 8080:80 nginx
```

8080 = port on YOUR machine (host)

80 = port INSIDE the container

อ่านคำสั่งให้เข้าใจ

```
docker run -p 8080:80 nginx
```

docker run สร้างและเริ่ม container ใหม่

-p 8080:80 เปิด port **8080** บนเครื่องเรา แล้วส่งต่อไป port **80** ในกล่อง

nginx image ที่จะใช้ (Nginx ฟังที่ port 80 เป็นค่าเริ่มต้น)

เปิดเบราว์เซอร์ที่ `http://localhost:8080` → จะเห็นหน้า **Welcome to nginx!**

จำง่าย ๆ : เลข **ซ้าย = ของเรา** (พิมพ์ในเบราว์เซอร์) · เลข **ขวา = ของกล่อง** (แอปข้างในฟังอยู่)

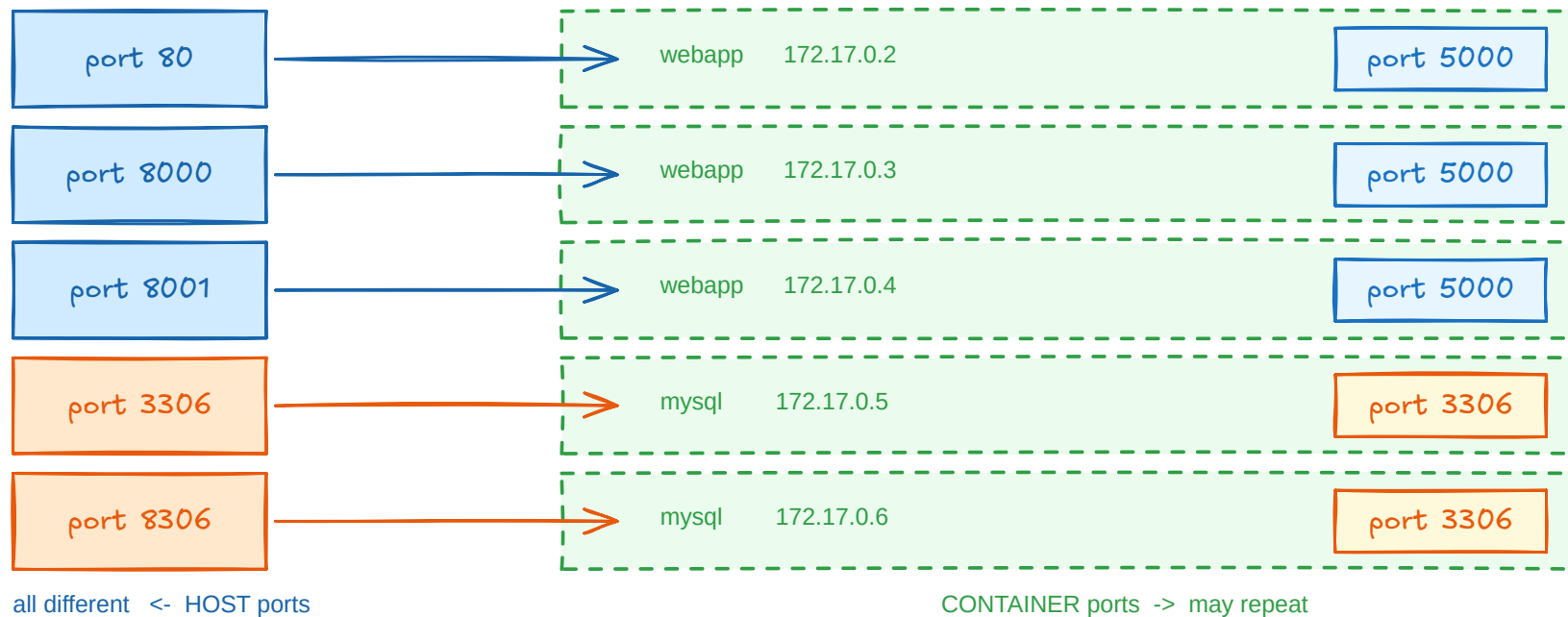
รันหลาย container พร้อมกันบนเครื่องเดียว

ขั้นที่ 1 — ดูผลลัพธ์ก่อนว่าหน้าตาเป็นอย่างไร

One Host, Many Containers

on the host (outside the containers)

each container is its own box



ภาพนี้เกิดจากคำสั่ง `docker run -p` เพียง **5 บรรทัด** — สไลด์ถัดไปจะไล่ดูทีละบรรทัดว่า **บรรทัดไหนสร้างแถวไหน**

5 คำสั่งที่ทำให้เกิดภาพเมื่อگی

ขั้นที่ 2 — บรรทัดบนสุดคือแถวบนสุดของภาพก่อนหน้า ไส้ลงมาตามลำดับ

```
docker run -p 80:5000 myname/simple-webapp # -> webapp 172.17.0.2 : 5000
docker run -p 8000:5000 myname/simple-webapp # -> webapp 172.17.0.3 : 5000
docker run -p 8001:5000 myname/simple-webapp # -> webapp 172.17.0.4 : 5000
docker run -p 3306:3306 mysql # -> mysql 172.17.0.5 : 3306
docker run -p 8306:3306 mysql # -> mysql 172.17.0.6 : 3306
```

อ่านเลขสองฝั่งของ -p ตามแนวตั้ง — ฝั่งซ้าย **ไม่ซ้ำกันเลยสักตัว** ส่วนฝั่งขวา **ซ้ำได้** นั่นคือกฎทั้งหมด (เลข IP ในบรรทัดคอมเมนต์อ่านตามภาพสไลด์ก่อนหน้า — ของจริงบนเครื่องเรียนอยู่ในช่วง 172.18.0.x)

ถ้าฝาแฝด — ใช้ host port ซ้ำ

จะไม่ได้ container ตัวใหม่ แต่ได้ error

Bind for 0.0.0.0:<port> failed: port is already allocated — สไลด์ถัดไปคือของจริง

ทำไม container port ซ้ำได้

เพราะแต่ละ container มี **network stack ของตัวเอง** — port 5000 ของกล่องหนึ่ง ไม่ใช่ port 5000 ของอีกกล่อง

ทดลองกฎเหล็กจริง — 3 กล่อง 3 port

ผลลัพธ์จริงบนเครื่องเรียน ทั้งกรณีที่ถูกและกรณีที่ชนกัน

host port ต่างกัน -> สร้างได้ทุกตัว (แต่ container port เป็น 80 เหมือนกันหมด)

```
docker run -d -p 8000:80 --name web2 nginx
```

```
docker run -d -p 8001:80 --name web3 nginx
```

```
docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
2eddc160a563	nginx	"/docker-entrypoint..."	1 second ago	Up Less than a second	0.0.0.0:8001->80/tcp, [::]:8001->80/tcp	web3
05d055ec3f8f	nginx	"/docker-entrypoint..."	1 second ago	Up Less than a second	0.0.0.0:8000->80/tcp, [::]:8000->80/tcp	web2
765b603bbb1a	nginx	"/docker-entrypoint..."	2 seconds ago	Up 1 second	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	web1

ผ่าฝืนกฎ : ใช้ host port 8000 ซ้ำ

```
docker run -d -p 8000:80 --name web4 nginx
```

```
docker: Error response from daemon: failed to set up container networking: driver failed programming
```

```
external connectivity on endpoint web4 (d443e76eed0a...): Bind for 0.0.0.0:8000 failed: port is already allocated
```

```
EXIT CODE: 125
```

IP ของแต่ละกล่อง (ของจริง)

web1 → 172.18.0.2

web2 → 172.18.0.3 · web3 → 172.18.0.4

ทดสอบทั้ง 3 port

curl -I localhost:8080 → **200 OK**

curl -I localhost:8000 → **200 OK**

curl -I localhost:8001 → **200 OK**

web4 ไม่หายไปเอง

รันไม่สำเร็จ แต่ container ถูกสร้างค้างไว้ในสถานะ Created — เห็นได้จาก `docker ps -a` ต้อง `docker rm web4` เอง

ปฏิบัติการ : รัน Nginx พร้อม Port Mapping

- 1 เตรียมโฟลเดอร์ แล้ว clone โค้ดแล็บ (ทำครั้งเดียว ใช้ได้ทุกแล็บ)

```
mkdir -p ~/labwork ; cd ~/labwork
git clone https://github.com/Tuchsanai/DevTools.git
cd DevTools/03_Docker/01_Docker/002_LAB_Nginx_Port_Mapping
```

- 2 รัน Nginx แบบ foreground — terminal จะค้าง กด Ctrl+C เพื่อหยุด

```
docker run -p 8080:80 nginx
```

- 3 รันใหม่แบบ **detached** (`-d`) จะได้ prompt คืน แล้วสั่งคำสั่งอื่นต่อได้

```
docker run -d -p 8080:80 --name web1 nginx
```

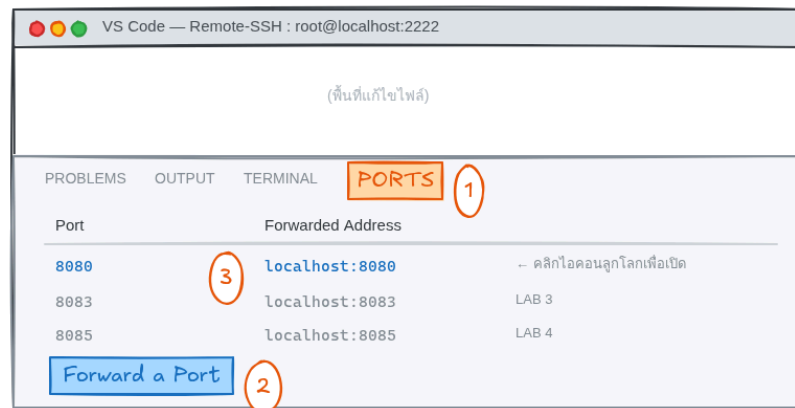
- 4 ทดสอบด้วย `curl http://localhost:8080` · ดู log ด้วย `docker logs web1` — จะเปิดดูในเบราว์เซอร์ ให้ forward port `8080` ที่แท็บ **PORTS** ของ VS Code ก่อน (วิธีทำอยู่ในสไลด์ถัดไป)

โฟลเดอร์แล็บในรีโฟคือ 001_LAB_Docker_Run... 004_LAB_Node_Bulletin_Board — เลขโฟลเดอร์ตรงกับเลขแล็บ (แล็บนี้อยู่ใน 002_LAB_Nginx_Port_Mapping) · เลิกใช้แล้วลบด้วย `docker rm -f web1`

เปิดดูในเบราว์เซอร์ — Forward Port ใน VS Code

port ของเว็บเปิดอยู่ในเครื่องเรียน — ทำ SSH tunnel ออกมาก่อน แล้วลบ tunnel เมื่อเลิกใช้

Forward Port ใน VS Code — เปิดหน้าเว็บของเว็บในเบราว์เซอร์



ขั้นตอน (ทำใน VS Code ที่ต่อ Remote-SSH อยู่)

- 1) เปิดแท็บ PORTS (แถวเดียวกับ TERMINAL)
- 2) กดปุ่ม Forward a Port
- 3) พิมพ์เลข port ของเว็บ เช่น 8080 แล้วกด Enter
- 4) เปิด `http://localhost:8080` ในเบราว์เซอร์

ทำไมต้อง forward? — port 8080 เปิดอยู่ใน "เครื่องเรียน devtools" ไม่ใช้บนเครื่องเราโดยตรง VS Code จะสร้าง SSH tunnel ให้อัตโนมัติ พอ forward แล้ว localhost:8080 บนเครื่องเรา จึงเข้าถึงเว็บในเว็บได้

Port ของแต่ละเว็บ : LAB 2 → 8080 · LAB 3 → 8083 · LAB 4 → 8085



เส้นทางของ request : เบราว์เซอร์ → SSH tunnel ของ VS Code → port 8080 ในเครื่องเรียน → nginx ในกล่อง (port 80)

ทางเลือก (ไม่ใช่ VS Code) : เปิด terminal ใหม่บนเครื่องเรา แล้ว **ssh เข้าไปยัง port ภายใน** ด้วย `ssh -L 8080:localhost:8080 root@localhost -p 2222` — ผูก port 8080 บนเครื่องเรา เข้ากับ port 8080 ในเครื่องเรียนโดยตรง

ทดลองเสร็จ ให้ลบ tunnel ทุกครั้ง — แบบ VS Code : กด PORTS คลิกขวาที่ port → **Stop Forwarding Port** · แบบ ssh -L : พิมพ์ `exit` หรือกด `Ctrl+D` ใน session นั้น

ผลลัพธ์ที่ควรได้

ผลลัพธ์จริงบนเครื่องเรียน — ตรวจสอบจาก terminal ได้โดยไม่ต้องเปิดเบราว์เซอร์

```
docker ps # (ตัดบางคอลัมน์)
CONTAINER ID   IMAGE     STATUS   PORTS                               NAMES
765b603bbb1a   nginx     Up Less than a second   0.0.0.0:8080->80/tcp   web1

curl -I http://localhost:8080
HTTP/1.1 200 OK
Server: nginx/1.31.3
Date: Mon, 10 Aug 2026 15:01:13 GMT
Content-Type: text/html
Content-Length: 896

curl http://localhost:8080
<title>Welcome to nginx!</title>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully
installed and working. ...
```

Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to is.com/nginx.

Thank you for using nginx.

เปิดเบราว์เซอร์ที่ :8080 ได้หน้าเดียวกับที่ curl ดังมา

รันแบบ **foreground** terminal จะค้าง — และทุกครั้งที่มีการเปิดหน้าเว็บ จะเห็น access log เด้งขึ้นมาสด ๆ

```
# log ที่เด้งขึ้นตอน curl เข้าไป (ของจริง — ดูย้อนหลังได้ด้วย docker logs web1)
172.18.0.1 - - [10/Aug/2026:15:01:13 +0000] "GET / HTTP/1.1" 200 896 "-" "curl/8.5.0" "-"
```

LAB 3

Volume Mapping

ทำให้ข้อมูลอยู่รอดแม้ลบ container
และแก้ไฟล์บนเครื่องแล้วเห็นผลทันที

ปัญหา : ลบ container = ข้อมูลหาย

เขียนไฟล์ลงในกล่อง แล้วลบกล่องทิ้ง — ลองดูว่าเกิดอะไรขึ้น

```
# 1) สร้างกล่อง แล้วเขียนไฟล์ลงไปข้างใน
docker run -d --name box1 nginx
docker exec box1 sh -c "echo ข้อมูลสำคัญของเรา > /note.txt"
docker exec box1 cat /note.txt      ข้อมูลสำคัญของเรา      # ไฟล์อยู่ครบ

# 2) ลบกล่องทิ้ง แล้วสร้างกล่องใหม่จาก image เดิม
docker rm -f box1
docker run -d --name box2 nginx
docker exec box2 cat /note.txt      cat: /note.txt: No such file or directory
```

เกิดอะไรขึ้น

ทุกอย่างที่เขียนลงไป**ในกล่อง** อยู่บนชั้นชั่วคราวของกล่องนั้น — **ลบกล่อง = ลบข้อมูล**
ถ้าเป็นฐานข้อมูล ก็คือข้อมูลสูญหายทั้งหมด

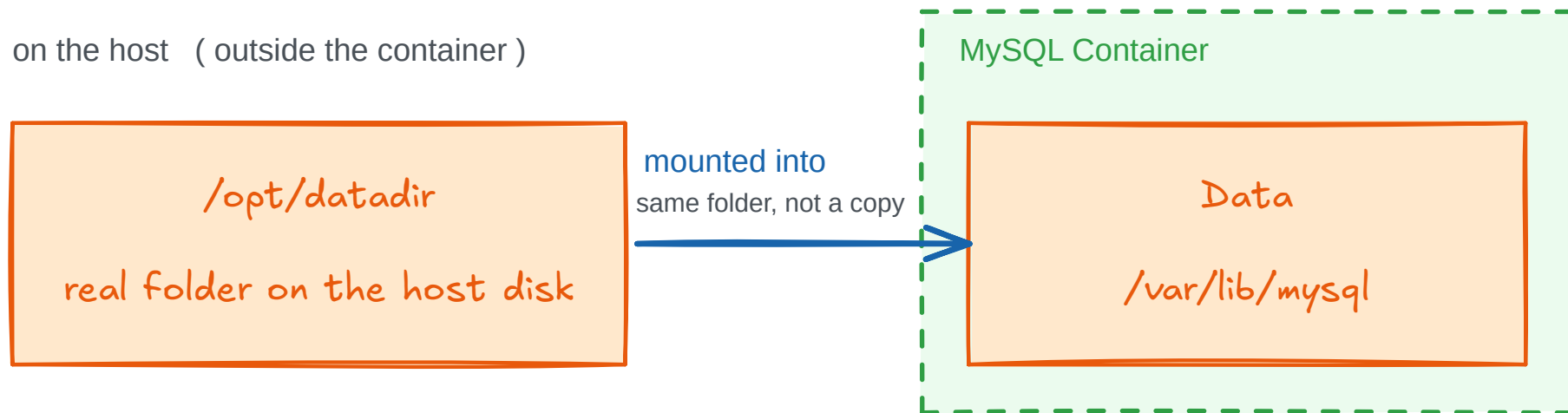
ทางแก้ : Volume Mapping

ผูกโฟลเดอร์จริงบน**เครื่องเรา**เข้ากับ path ในกล่อง → ข้อมูลถูกเขียนลงดิสก์ของเรา ไม่ได้อยู่ในกล่อง **ลบกล่องที่รอบก็ไม่หาย**

Volume Mapping ทำงานอย่างไร

Volume Mapping -v HOST_PATH:CONTAINER_PATH

on the host (outside the container)



```
docker run -v /opt/datadir:/var/lib/mysql mysql
```

concept only - the real mysql image also needs `-e MYSQL_ROOT_PASSWORD=...` and `-d`

Delete the container -> the folder and its data stay on the host (persistent)

| รูปแบบคำสั่ง -v

```
docker run -v <HOST_PATH>:<CONTAINER_PATH>[:ro] <image>
```

HOST_PATH โฟลเดอร์จริงบนเครื่องเรา (ต้องเป็น absolute path — ใช้ \${PWD} ช่วยได้)

CONTAINER_PATH path ในกล่องที่จะถูก **แทนที่** ด้วยโฟลเดอร์ข้างบน

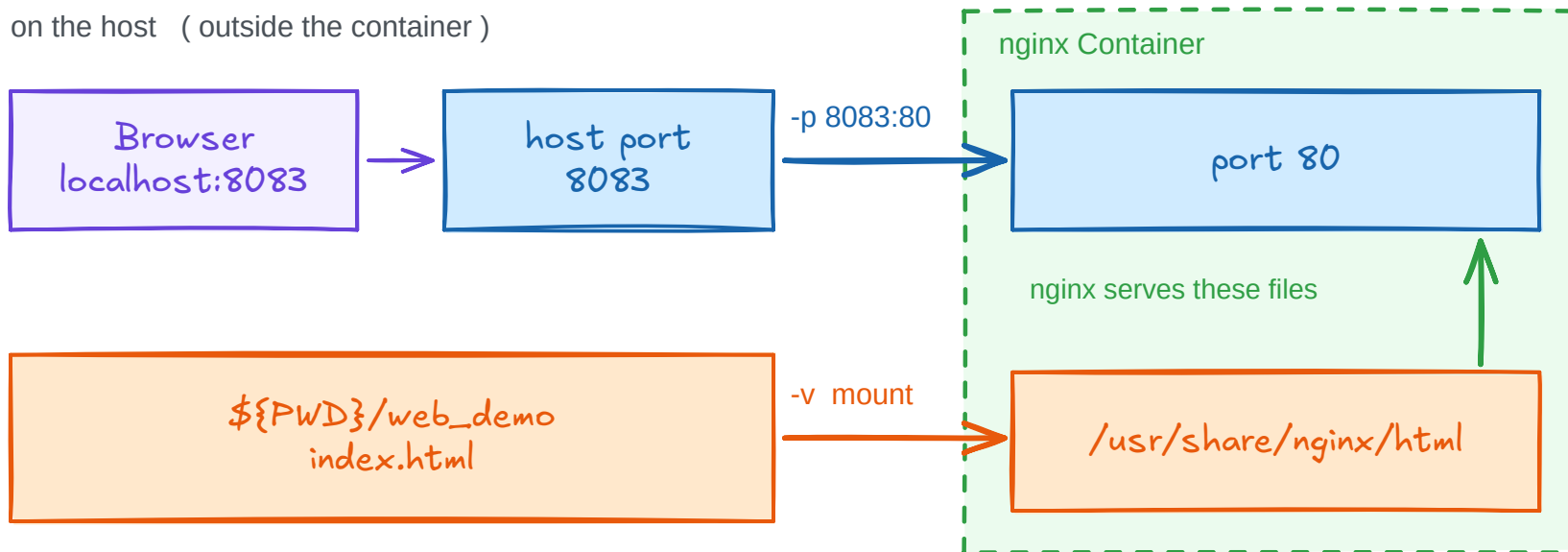
:ro read-only — ให้ container **อ่านได้อย่างเดียว** **แก้ไขไม่ได้** (ไม่ใส่ก็เขียนได้)

ระวัง : ของเดิมที่อยู่ใน CONTAINER_PATH จะถูก **บงกั** ด้วยโฟลเดอร์จาก host ทั้งหมด — ไม่ใช้การรวมกัน

รวม Volume และ Port เข้าด้วยกัน

LAB 4 : Nginx + Volume + Port

on the host (outside the container)



```
docker run -d -p 8083:80 -v ${PWD}/web_demo:/usr/share/nginx/html:ro nginx
```

:ro = the container cannot write to it - you can still edit index.html on the host

Edit index.html on the host -> refresh the browser -> the page changes, no restart needed

ปฏิบัติการ

- 1 เข้าไปที่ไดเรกทอรีที่ clone ไว้ตอน LAB 2

```
cd ~/labwork/DevTools/03_Docker/01_Docker
```

- 2 เข้าโฟลเดอร์ของแล็บนี้ (ถ้ายังไม่เคย clone ให้ clone ก่อนเหมือน LAB 2)

```
cd 003_LAB_Nginx_Volume_Port_Mapping
ls # จะเห็นโฟลเดอร์ web_demo ที่มี index.html ของเรา
```

- 3 รัน Nginx พร้อม volume + port mapping

```
docker run -d -p 8083:80 -v ${PWD}/web_demo:/usr/share/nginx/html:ro nginx
```

- 4 ทดสอบที่ `http://localhost:8083` → เห็นหน้าเว็บของเราเอง ไม่ใช่หน้า default — จะเปิดในเบราว์เซอร์ให้ forward port 8083 เหมือนที่ทำใน LAB 2 (แก้ **PORTS** หรือ `ssh -L 8083:localhost:8083 root@localhost -p 2222`) และ **au tunnel** เมื่อเลิกใช้

รันจริงได้หน้าเว็บของวิชาเรา (Tailwind CSS + ลูกเล่น) จาก web_demo/index.html — ไม่ใช่หน้า **Welcome to nginx!** ของ image อีกต่อไป

แยกอธิบายทีละ flag

```
docker run -d -p 8083:80 -v ${PWD}/web_demo:/usr/share/nginx/html:ro nginx
```

Flag	ความหมาย
-d	detached — รันเบื้องหลัง คั่น prompt ให้ทันที ไม่ค้าง terminal
-p 8083:80	เปิด port 8083 บน host ส่งต่อไป port 80 ในกล่อง
-v \${PWD}/web_demo: /usr/share/nginx/html:ro	เอาโฟลเดอร์ <code>web_demo</code> ในไดเรกทอรีปัจจุบัน ไปวางแทน web root ของ Nginx แบบอ่านอย่างเดียว
nginx	image ที่ใช้

```
# พิสูจน์ว่า web root ในกล่องถูกแทนที่จริง — เหลือแค่ไฟล์ของเรา
docker exec <container> ls -l /usr/share/nginx/html
total 16
-rwxr-xr-x 1 root root 12312 Aug 10 15:31 index.html  # ไม่มี 50x.html ของ nginx แล้ว

# เทียบกับ nginx ที่ไม่ได้ผูก volume — ของเดิมยังอยู่ครบ
docker run --rm nginx ls -l /usr/share/nginx/html
total 8
-rw-r--r-- 1 root root 497 Jul 15 16:03 50x.html
-rw-r--r-- 1 root root 896 Jul 15 16:03 index.html
```

ทดสอบ : แก้ไฟล์แล้วเห็นผลทันที

ผลจริง — ไม่ได้ restart container เลยสักครั้ง

ก่อนแก้ไฟล์

```
curl -s http://localhost:8083 | grep -o "Welcome to Demo nginx Website"    Welcome to Demo nginx Website
```

แก้ index.html บนเครื่องเรียน (ไม่แตะ container) แล้ว curl ซ้ำ — ไม่ได้ restart อะไรเลย

```
sed -i "s|Welcome to Demo nginx Website|สวัสดี Docker|" web_demo/index.html
```

```
curl -s http://localhost:8083 | grep -o "สวัสดี Docker"                สวัสดี Docker
```

SOFTWARE DEVELOPMENT TOOLS AND ENVIRONMENTS
WEEK 8 - DOCKER
LAB 3 - VOLUME MAPPING



สวัสดี Docker

หน้านี้ **ไม่ได้** อยู่ใน image ของ nginx — มันคือไฟล์ `web_demo/index.html` บนเครื่องเรา ที่ถูก
ผูกเข้าไปในกล่องด้วย `-v`

ลองกดดู
สลับไปมอดปีด/สว่าง
16:18:50



Port Mapping

-p 8083:80 ส่ง port 8083 ของเครื่องเรา เข้า
port 80 ที่ Nginx ฟังอยู่ภายใน



Volume Mapping

--v \${PWD}/web_demo วางไฟล์ของเรา
กับ web root ของ Nginx ที่จุด



Read-only :ro

กล่องงานไฟล์เราได้ แต่เขียนไม่ได้ — ลอส่ง
เขียนจะได้ Read-only file system

คำสั่งเดียวที่ทำให้หน้านี้ขึ้นมา

```
docker run -d -p 8083:80 \
-v ${PWD}/web_demo:/usr/share/nginx/html:ro \
nginx
```

READER

หัวข้อด้านบนเปลี่ยนเป็น **สวัสดี Docker** กันที่ทริเฟรช — แก้โค้ดบนเครื่อง เห็นผลในกล่องทันที ไม่ต้อง build ใหม่

พิสูจน์ : ro และการอยู่รอดของข้อมูล

สองข้อสรุปสำคัญของ volume — ทดสอบจริงทั้งคู่

```
# 1) :ro ทำให้ container เขียนทับไฟล์ของเราไม่ได้จริง
docker exec <container> sh -c "echo test >> /usr/share/nginx/html/index.html"
sh: 1: cannot create /usr/share/nginx/html/index.html: Read-only file system
EXIT CODE: 2

# 2) ลบ container ที่ แล้วดูไฟล์บนเครื่องเรียน
docker rm -f <container>
ls -l web_demo/index.html
-rwxr-xr-x 1 root root 12312 Aug 10 22:31 web_demo/index.html # ยังอยู่ครบ
grep -o "สวัสดี Docker" web_demo/index.html
สวัสดี Docker # ข้อความที่แก้ก็ยังมีอยู่
```

:ro ป้องกันอะไร

ถ้าแอปในกล่องถูกเจาะ หรือเขียนไฟล์มั่ว ก็แก้ไฟล์ต้นฉบับบนเครื่องเราไม่ได้ — ปลอดภัยกว่าตอนเอาโค้ดไปแปะ

ข้อมูลอยู่ที่ไหนกันแน่

ไฟล์อยู่บน **ดิสก์ของ host** ตลอด กล่องแค่ "มอง" เข้าไป — ลบกล่องที่รอบข้อมูลก็ไม่หาย

LAB 4

Build Docker Image เอง

จากซอร์สโค้ด Node.js → Dockerfile → Image → Container

เส้นทางจากโค้ดสู่ Container

Dockerfile -> Image -> Container



```
docker build -t bulletinboard:1.0 .
```

the lone `.` at the end is the build context = the current directory (do not drop it)

```
docker run -p 8085:8080 -d --name bb bulletinboard:1.0
```

Build ONCE -> the same image can start many containers (give each a different `--name` and `-p`)

| ปฏิบัติการ : Node.js Bulletin Board

- 1 เข้าไปที่โฟลเดอร์แอป (ในรีโพที่ clone ไว้แล้ว)

```
cd ~/labwork/DevTools/03_Docker/01_Docker  
cd 004_LAB_Node_Bulletin_Board/bulletin-board-app
```

- 2 ดูให้แน่ใจว่ามี `Dockerfile` อยู่ในโฟลเดอร์นี้

```
ls # Dockerfile LICENSE app.js backend fonts index.html package.json server.js site.css
```

- 3 Build image

```
docker build -t bulletinboard:1.0 .
```

- 4 รัน container จาก image ที่เพิ่ง build

```
docker run -p 8085:8080 -d --name bb bulletinboard:1.0
```

อ่าน Dockerfile ทีละบรรทัด

```
FROM node:current-slim      # 1. เลือก base image (Node.js รุ่นเล็ก)

WORKDIR /usr/src/app        # 2. กำหนด working directory ในกล่อง

COPY package.json .         # 3. คัดลอกเฉพาะ package.json เข้ามาก่อน
RUN npm install              # 4. ติดตั้ง dependencies

EXPOSE 8080                  # 5. ประกาศว่า container ฟังที่ port 8080
CMD [ "npm", "start" ]      # 6. คำสั่งที่รันเมื่อ container เริ่มทำงาน

COPY . .                     # 7. คัดลอกซอร์สโค้ดที่เหลือทั้งหมด
```

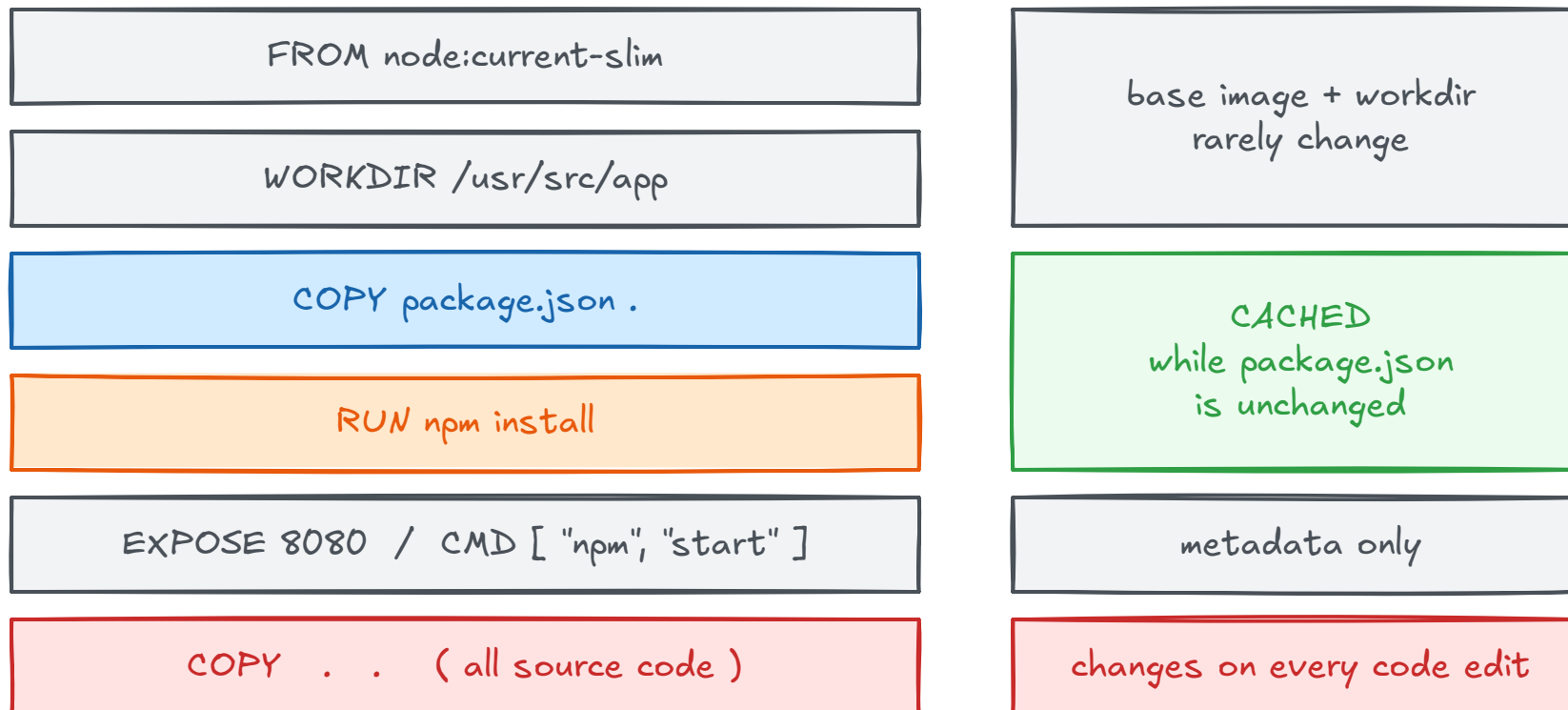
EXPOSE ไม่ได้เปิด port ให้จริง — เป็นเพียงการ "ประกาศเจตนา" ว่าแอปในกล่องฟังที่ port ไหน การเปิดจริงต้องใช้ -p ตอน docker run

ข้อสังเกต : ไฟล์นี้วาง COPY . . ไว้หลัง CMD ซึ่งยังทำงานได้ปกติ (เพราะ CMD เป็นแค่การประกาศ ไม่ใช้การรันตอน build) แต่ตามธรรมเนียมที่ดีควรวางไว้ก่อน EXPOSE/CMD

ทำไมต้อง COPY package.json ก่อน?

Dockerfile Layers & Build Cache

Rule : change ONE layer -> every layer BELOW it must be rebuilt



So put the layer that changes most often (COPY . .) LAST -> npm install stays cached

ผลการ docker build จริง

Dockerfile 6 บรรทัด → 5 ขั้นตอน build ที่ Docker ทำให้

```
docker build -t bulletinboard:1.0 .
#1 [internal] load build definition from Dockerfile    transferring dockerfile: 164B    DONE 0.1s
#2 [internal] load metadata for docker.io/library/node:current-slim    DONE 2.4s
#3 [internal] load .dockerignore    DONE 0.0s    #4 load build context    38.10kB    DONE 0.1s
#5 [1/5] FROM docker.io/library/node:current-slim@sha256:4ebb5ace66f1...    DONE 14.6s
#6 [2/5] WORKDIR /usr/src/app    DONE 0.1s
#7 [3/5] COPY package.json .    DONE 0.1s
#8 [4/5] RUN npm install
#8 8.277 added 115 packages, and audited 116 packages in 8s    DONE 8.4s
#9 [5/5] COPY . .    DONE 0.1s
#10 naming to docker.io/library/bulletinboard:1.0 done    DONE 1.5s
```

ตัวเลข [1/5] ... [5/5] คือ **คำสั่งใน Dockerfile ที่สร้าง layer** — EXPOSE และ CMD ไม่นับ เพราะเป็นแค่ metadata ไม่ได้สร้าง layer

ขั้นที่กินเวลาที่สุดคือ FROM ที่ต้องโหลด base image (**14.6 วินาที**) และ RUN npm install (**8.4 วินาที**) — รวมทั้ง build ประมาณ **25 วินาที** จำไว้ แล้วดูสไลด์ถัดไป

พิสูจน์ Layer Cache — build รอบสอง

แก้ไขโค้ด (ไม่แตะ package.json) แล้ว build ใหม่

```
# แก้ index.html (ไม่แตะ package.json) แล้วสั่ง build ใหม่
sed -i 's/Welcome to the Bulletin Board/Welcome to the Bulletin Board v1.1/' index.html
docker build -t bulletinboard:1.1 .
#5 [1/5] FROM docker.io/library/node:current-slim@sha256:4ebb5ace... DONE 0.0s
#6 [3/5] COPY package.json . CACHED
#7 [2/5] WORKDIR /usr/src/app CACHED
#8 [4/5] RUN npm install CACHED # <- ประหยัดไป 8.4 วินาที
#9 [5/5] COPY . . DONE 0.0s
#10 naming to docker.io/library/bulletinboard:1.1 done DONE 0.3s
# build รอบแรก ~25 วินาที -> รอบนี้เหลือไม่ถึง 2 วินาที
```

ถ้าเขียน COPY . . ก่อน RUN npm install

แก้ไขโค้ดแค่ตัวอักษรเดียว → layer COPY เปลี่ยน → ทุก layer หลังจาก
นั้นถูกยกเลิก cache → npm install ต้องรันใหม่ทุกครั้ง

เขียน COPY package.json ก่อน (แบบนี้)

ตราใบที่ package.json ไม่เปลี่ยน layer npm install ก็
CACHED ตลอด — build รอบถัดไปเหลือไม่ถึงวินาที

กฎง่าย ๆ : ใส่สิ่งที่ **เปลี่ยนน้อยที่สุด** ไว้บนสุดของ Dockerfile และสิ่งที่ **เปลี่ยนบ่อยที่สุด** ไว้ล่างสุด

แยกอธิบายคำสั่ง build และ run

```
docker build -t bulletinboard:1.0 .
```

ส่วน	ความหมาย
<code>docker build</code>	สั่งสร้าง image จาก Dockerfile
<code>-t bulletinboard:1.0</code>	ตั้งชื่อ (tag) image เป็น <code>bulletinboard</code> เวอร์ชัน <code>1.0</code>
<code>.</code>	build context = โดเรกทอรีปัจจุบัน (ที่มี Dockerfile อยู่)

```
docker run -p 8085:8080 -d --name bb bulletinboard:1.0
```

Flag	ความหมาย
<code>-p 8085:8080</code>	host <code>8085</code> → container <code>8080</code> (แอป Node.js ฟังที่ 8080)
<code>-d</code>	รันเบื้องหลัง
<code>--name bb</code>	ตั้งชื่อ container ว่า <code>bb</code> เพื่ออ้างอิงง่าย ไม่ต้องจำ ID

ผลลัพธ์หลัง docker run — ของจริง

```
docker run -p 8085:8080 -d --name bb bulletinboard:1.0
fb1694ca74a7f3b7e486fce2e1e6efea8583be626369dee34912409280575f78 # container ID

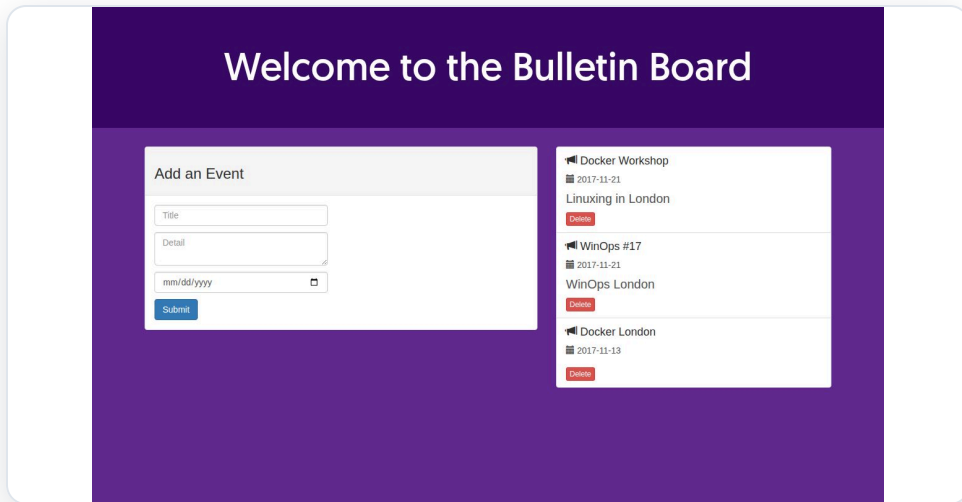
docker images
IMAGE                ID                DISK USAGE  CONTENT SIZE  EXTRA
bulletinboard:1.0    0924eadc8511      399MB       94MB         U      # image ที่เราสร้างเอง

docker ps
CONTAINER ID  IMAGE                COMMAND                  CREATED        STATUS        PORTS                NAMES
fb1694ca74a7  bulletinboard:1.0    "docker-entrypoint.s..." 7 seconds ago  Up 7 seconds  0.0.0.0:8085->8080/tcp, ...  bb

docker logs bb
> vue-event-bulletin@1.0.0 start
> node server.js
Magic happens on port 8080... # แอปฟังที่ 8080 ในกล่อง
```

ข้อความ **Magic happens on port 8080** คือหลักฐานว่าแอปในกล่องขึ้นแล้ว — ถ้าหน้าเว็บไม่ขึ้น ให้ดู `docker logs bb` ก่อนเสมอ

ทดสอบแอป และคำสั่งที่ใช้บ่อยหลัง build



เปิดดูโดย forward port 8085 (เก็บ **PORTS** หรือ `ssh -L 8085:localhost:8085 root@localhost -p 2222`) — เสร็จแล้วลบ **tunnel** ทุกครั้ง

```
curl -s http://localhost:8085 | head -30
<title>Bulletin Board</title>
<h1>Welcome to the Bulletin Board</h1>
```

```
curl -s http://localhost:8085/api/events
[{"id":1,"title":"Docker Workshop","detail":
  "Linuxing in London ","date":"2017-11-21"}, ...]
```

```
docker exec bb sh -c "node -v; pwd; ls"
v26.7.0
/usr/src/app
Dockerfile LICENSE app.js backend fonts index.html
node_modules package-lock.json package.json
server.js site.css
```

node_modules ถูกสร้างขึ้นใน**ในกล่อง**ตอน RUN `npm install` — ไม่ได้ก๊อปปมาจากเครื่องเรา

```
docker logs -f bb # ตามดู log สด ๆ      docker stop bb # หยุด
docker rm bb      # ลบ container          docker rmi bulletinboard:1.0 # ลบ image
```

02

สรุป

สรุปคำสั่งของแต่ละแล็บ · ตาราง port ที่ใช้ · การล้างระบบ

| สรุปคำสั่ง — SETUP · LAB 1

เตรียมเครื่อง แล้วเริ่มรัน container ตัวแรก

SETUP — เปิดเครื่องเรียนและตรวจสอบ

```
docker run -dit --name devtools \
  --privileged -p 2222:22 \
  tuchsanai/devtools:2569_1
# เปิดเครื่องเรียน

ssh root@localhost -p 2222
# เข้าไปข้างใน ( password : passwd )

docker --version
docker compose version
# ตรวจสอบว่าพร้อมใช้งาน
```

LAB 1 — รัน และดูสถานะ

```
docker run nginx
# โหลดให้ถ้ายังไม่มี แล้วรันเลย

docker pull nginx
# โหลดอย่างเดียว ยังไม่รัน

docker run ubuntu sleep 5
# รันพร้อมสั่งคำสั่งให้ทำ

docker ps          # ที่กำลังทำงาน
docker ps -a       # ทั้งหมด รวมทั้งที่หยุดแล้ว
docker images      # image ที่มีในเครื่อง
docker stop / start / rm # จัดการกล่อง
```

| สรุปคำสั่ง — LAB 2 · LAB 3

เปิดให้เข้าถึงจากภายนอก และผูกไฟล์บนเครื่องเข้ากับกล่อง

LAB 2 — Port Mapping

```
docker run -p 8080:80 nginx
#           ^^^^ ^^
#           host  container
```

เลขซ้ายคือ port ที่เราพิมพ์ในเบราว์เซอร์ เลขขวาคือ port ที่แอปในกล่องฟังอยู่

host port **ห้ามซ้ำ** · จะเปิดดูในเบราว์เซอร์ ต้อง forward port ใน VS Code ก่อน

LAB 3 — Volume Mapping

```
docker run -d -p 8083:80 \
-v ${PWD}/web_demo:/usr/share/nginx/html:ro \
nginx
```

- d** รันเบื้องหลัง ไม่ค้าง terminal
- v** เอาไฟล์เดสก์ท็อปไปวางในกล่อง
- :ro** กล่องอ่านได้อย่างเดียว เขียนไม่ได้

| สรุปคำสั่ง — LAB 4 · การจัดการ

สร้าง image ของตัวเอง แล้วดูแล container ที่รันอยู่

LAB 4 — สร้าง image ของตัวเอง

```
docker build -t bulletinboard:1.0 .  
# สร้าง image จาก Dockerfile  
# จุดท้ายสุด = โพลเดอร์ปัจจุบัน  
  
docker run -p 8085:8080 -d \  
  --name bb bulletinboard:1.0  
# รัน container จาก image ที่เพิ่งสร้าง  
# --name ตั้งชื่อไว้เรียกใช้ง่าย
```

ดูแล container ที่รันอยู่

```
docker logs bb          # ดู log  
docker logs -f bb      # ตามดูสด ๆ  
docker exec -it bb sh  
# เข้าไปใน shell ของกล่อง  
  
docker stop bb          # หยุด  
docker rm bb            # ลบ container  
docker rmi bulletinboard:1.0  
# ลบ image
```

หน้าเว็บไม่ขึ้น? ให้ดู docker logs ก่อนเสมอ